tempcolor[rgb]blacktempcolor[rgb]white

Markov Chain Monte Carlo Methods, Part 1

JEFF GILL
Distinguished Professor
Departments of Government and Mathematics & Statistics
American University

Status

- This is the official start of the second half of the Multilevel Analysis sequence.

- Our methodological roadmap:

tool	*capabilities*	*limitations*
using `lmer` in `R`	basic models, good understanding	normally distributed group levels, limited link functions
`jags` and `bugs`	builds on `R` , multiple hierarchies, no distributional restrictions intuitive modeling language	none

- `jags` and `bugs` are completely free implementations of *Markov chain Monte Carlo*, which revolutionized statistics starting in 1990.

Bureaucratic Politics Example

- Contains *every* federal political appointee to full-time positions requiring Senate confirmation from November, 1964 through December, 1984 (collected by Mackenzie and Light, ICPSR Study Number 8458, Spring 1987).

- The survey queries various aspects of the Senate confirmation process, acclamation to running an agency or program, and relationships with other functions of government.

- The authors needed to preserve anonymity so they embargoed some variables and randomly sampled 1,500 down to 512.

- These latter issues are dealt with in Gill and Casella (JASA 2009).

Bureaucratic Politics Example

- **Outcome Variable:** `stress` as a surrogate measure for self-perceived effectiveness and job-satisfaction, measured as a five-point scale from "not stressful at all" to "very stressful."

- **Explanatory Variables:**
  - `Government Experience`,
  - `Ideology`,
  - `Committee Relationship`,
  - `Career.Exec-Compet`,
  - `Career.Exec-Liaison/Bur`,
  - `Career.Exec-Liaison/Cong`,
  - `Career.Exec-Day2day`,
  - `Career.Exec-Diff`,
  - `Confirmation Preparation`,
  - `Hours/Week`,
  - `President Orientation`.

Ordered Logit Model

- A Bayesian random effects specification for ordered survey outcomes, so latent variable thresholds for $\mathbf{Y}$ are assumed on the ordering:

$$\mathbf{U}_i: \ \theta_0 \underset{c=1}{\Longleftrightarrow} \theta_1 \underset{c=2}{\Longleftrightarrow} \theta_2 \underset{c=3}{\Longleftrightarrow} \theta_3 \ldots \theta_{C-1} \underset{c=C}{\Longleftrightarrow} \theta_C$$

- The vector of (unseen) utilities across individuals in the sample, $\mathbf{U}$, is determined by a linear additive specification of explanatory variables: $\mathbf{U} = -\mathbf{X}'\boldsymbol{\gamma} + \boldsymbol{\eta}$, where $\boldsymbol{\gamma} = [\gamma_1, \gamma_2, \ldots, \gamma_p]$ does not depend on the θ_j, and $\boldsymbol{\eta} \sim F_{\boldsymbol{\eta}}$.

- This means that the probability that individual i in the sample is observed to be in category r or lower is:

$$P(\mathbf{Y}_i \leq r|\mathbf{X}_i) = P(\mathbf{U}_i \leq \theta_r) = P(\boldsymbol{\eta} \leq \theta_r + \mathbf{X}_i'\boldsymbol{\gamma}) = F_{\boldsymbol{\eta}_i}(\theta_r + \mathbf{X}_i'\boldsymbol{\gamma}),$$

  which is differently signed than in `R`: "$logitP(Y <= k|x) = zeta_k - eta$" from the help page.

- Specifying a logistic distributional assumption on the errors and adding the random effect term produces this logistic cumulative specification for the whole sample:

$$F_{\boldsymbol{\eta}}(\theta_r + \mathbf{X}'\boldsymbol{\gamma} + \mathbf{b}) = P(\mathbf{Y} \leq r|\mathbf{X}) = [1 + \exp(-\theta_r - \mathbf{X}'\boldsymbol{\gamma} + \mathbf{b})]^{-1}$$

Model Priors

- The software we will use is for Bayesian hierarchial models, meaning that we have to stipulate an assumed distribution for the unknown parmaeters *before* observing and analyzing the data.

- Most social scientists give fairly vague prior statements as a way not to worry about this.

- Prior distributions are either semi-informed or skeptical:

$p(\gamma_k) \sim \mathcal{N}(\mu_{\gamma_k}, \sigma^2_\gamma),\ k = 1, \ldots, p$	for each of the p explanatory variables,
$p(\theta_j) \sim \mathcal{N}(0, \sigma^2_\theta),\ j = 1, \ldots, C-1$	for the four latent variable thresholds,
$b_i \sim \mathcal{N}(0, \tau)$	for the random effects term,
$\tau \sim \mathcal{IG}(\delta_1, \delta_2)$	for the random effects hyperprior,

Model Posterior

- All this produces a posterior distribution according to:

$$\pi(\boldsymbol{\gamma}, \boldsymbol{\theta}|\mathbf{X}, \mathbf{Y}) \propto L(\boldsymbol{\gamma}, \boldsymbol{\theta}|\mathbf{X}, \mathbf{Y}) p(\boldsymbol{\theta}) p(\boldsymbol{\gamma}) p(b|\tau) p(\tau)$$

$$\propto \prod_{i=1}^{n} \prod_{j=1}^{C-1} \prod_{k=1}^{p} [\Lambda(\theta_j - \mathbf{X}_i'\boldsymbol{\gamma} + \mathbf{b}_i) - \Lambda(\theta_{j-1} - \mathbf{X}_i'\boldsymbol{\gamma} + \mathbf{b}_i)]^{z_{ij}}$$

$$\times \exp\left(-\frac{(\boldsymbol{\gamma}_k - \mu_{\gamma_k})^2}{2\sigma_\gamma^2} - \frac{\theta_j^2}{2\sigma_\theta^2} - \frac{b_i^2}{2\tau^2} - \frac{\delta_2}{\tau} \right) \tau^{-(\delta_1+1)}$$

  which is kind of ugly (and hard marginalize).

- While this form tells us everything we need to know about the *joint distribution* of the model parameters, we need to marginalize (integrate) it for every one of these parameters to create an informative regression table.

- Solution: Gibbs sampling (a type of MCMC), which is a special application of a special kind of stochastic process..

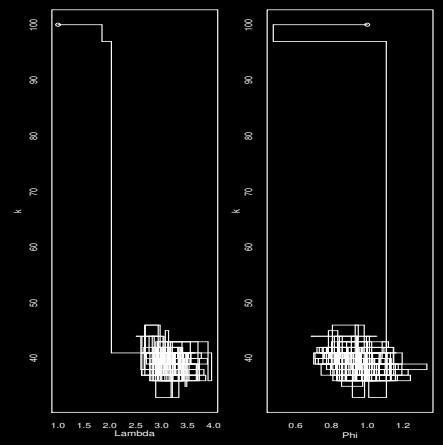
Technologies that have changed my life:

- gene sequencing
- iPhones
- online banking
- Gibbs sampling









Posterior Summary, Model for Survey of Political Executives

	Mean	Std.Err.	95% HPD Intervals
Explanatory Variables:			
Constant Term	1.20215	2.24169	
Government Experience	-0.65500	0.60664	
Ideology	-0.49140	0.32964	
Committee Relationship	1.14550	0.39131	
Career.Exec-Compet	1.73300	0.85088	
Career.Exec-Liaison/Bur	-2.81800	0.55620	
Career.Exec-Liaison/Cong	1.03675	0.49193	
Career.Exec-Day2day	-0.76595	0.38019	
Career.Exec-Diff	0.27780	0.29838	
Confirmation Preparation	1.02440	0.45753	
Hours/Week	-0.72720	0.42732	
President Orientation	1.94950	0.86787	
Threshold Intercepts:			
None|Little	-5.93500	1.85782	
Little|Some	-2.69250	1.59126	
Some|Significant	1.19300	1.52653	
Significant|Extreme	8.40450	2.10379	

$\hat{\sigma} = 6.04350$ (1.20325), dashed vertical line at zero.

Core BUGS Code for this Model

```
for (i in 1:N) {
  b[i] ~ dnorm(0.0, tau)
  mu[i] <- theta[1]
    + theta[2]*previous.empl[i]     + theta[3]*ideology[i]
    + theta[4]*senate.relat[i]     + theta[5]*confirm.prep[i]
    + theta[6]*hours.week[i]      + theta[7]*career.exec.compet[i]
    + theta[8]*career.exec.liason.bur[i] + theta[9]*career.exec.liason.cong[i]
    + theta[10]*career.exec.day2day[i]  + theta[11]*career.exec.diff[i]
    + theta[12]*president.orient[i]
    for (j in 1:Ncut) {logit(Q[i, j]) <- -(k[j] + mu[i] - b[i])} #cum prob lower than j
    # probability that response = j
    p[i, 1] <- max( min(1 - Q[i, 1], 1), 0)
    for (j in 2 : Ncut) { p[i,j] <- max( min(Q[i, j - 1] - Q[i, j],1), 0) }
    p[i, (Ncut+1)] <- max( min(Q[i, Ncut], 1), 0)
    stress[i] ~ dcat(p[i, ])
}
```

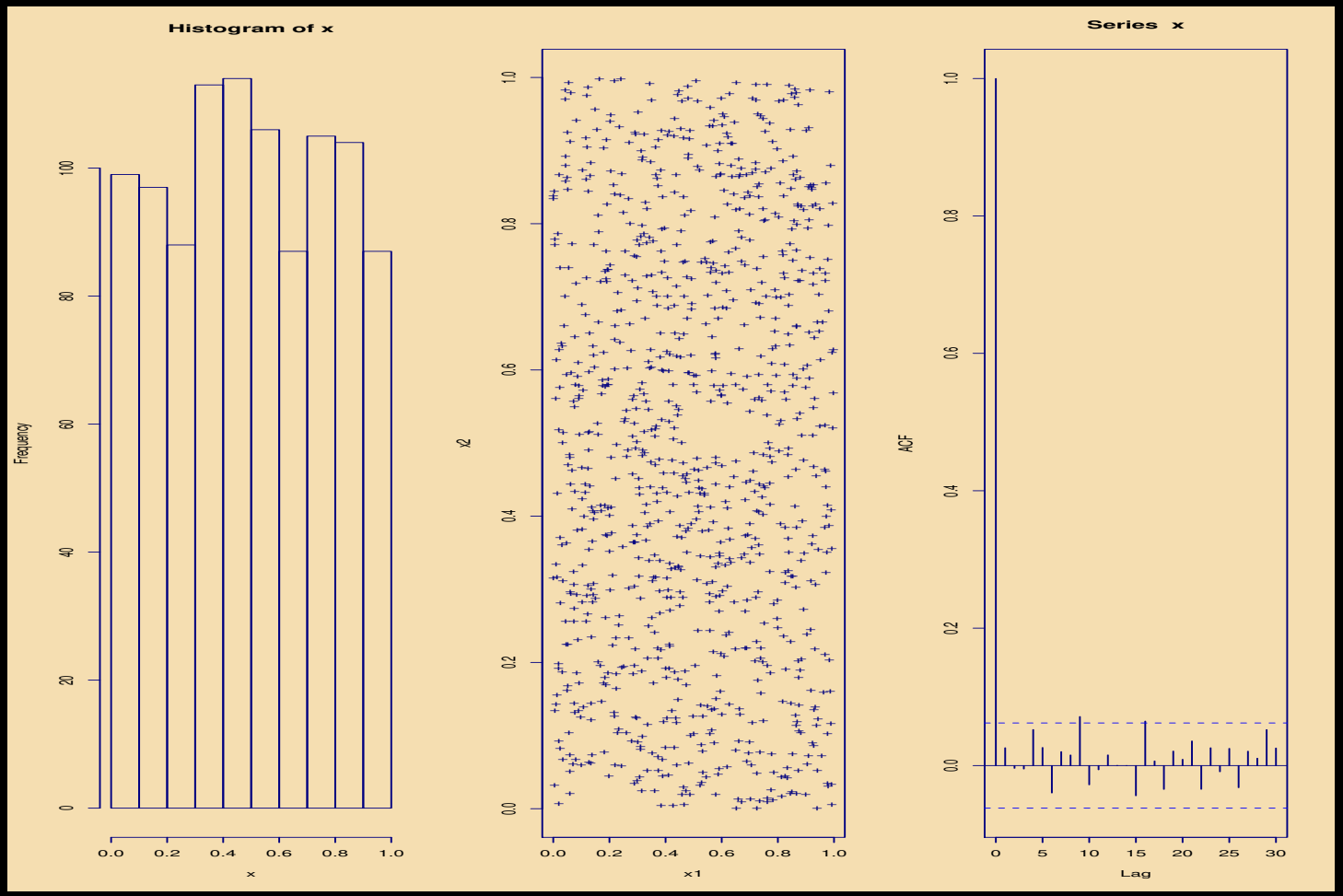
Monte Carlo Introduction

- Simulation work in applied statistics replaces analytical work with repetitious, low-level effort by the computer.

- If a distribution is difficult or impossible to manipulate analytically, then it is often possible to create a set of simulated values that share the same distributional properties, and describe the posterior by using empirical summaries of these simulated values.

- This process was termed *Monte Carlo simulation* by von Neumann and/or Ulam in the 1950s (although some accounts attribute the naming to Metropolis too) because it uses randomly generated values to perform calculations interest, perhaps reminiscent of expected value calculations in gambling.

Uniform Random Variables



Uniform Random Variables

- Surprisingly, all random number generation starts with generating $[0:1]$ continuous uniform random variables.

- Note that `runif` does not involve randomness per se: it is a deterministic sequence based on a random starting point. Setting the seed determines all the subsequent values.

- The `R` function set.seed can produce the same sequence:

```
set.seed(1)
runif(5)
[1] 0.2655087 0.3721239 0.5728534 0.9082078 0.2016819
set.seed(1)
runif(5)
[1] 0.2655087 0.3721239 0.5728534 0.9082078 0.2016819
set.seed(2)
runif(5)
[1] 0.0693609 0.8177752 0.9426217 0.2693818 0.1693481
```

Basic MC Principle: Make the Computer Do the Work

- Often you can get the right answer faster with `R` than by analytics.

- For instance, what is the 90% confidence interval for $N(6, 3)$?

```
n.data <- rnorm(1000000,6,3)
sort(n.data)[c(50000,950000)]
[1] 1.0697 10.9266
```

or

```
mean(n.data) + c(-1,1)*qnorm(0.95)*sd(n.data)
[1] 1.0662 10.9299
```

or

```
qnorm(c(0.05,0.95),6,3)
[1] 1.0654 10.9346
```

- Notice the slight simulation difference.

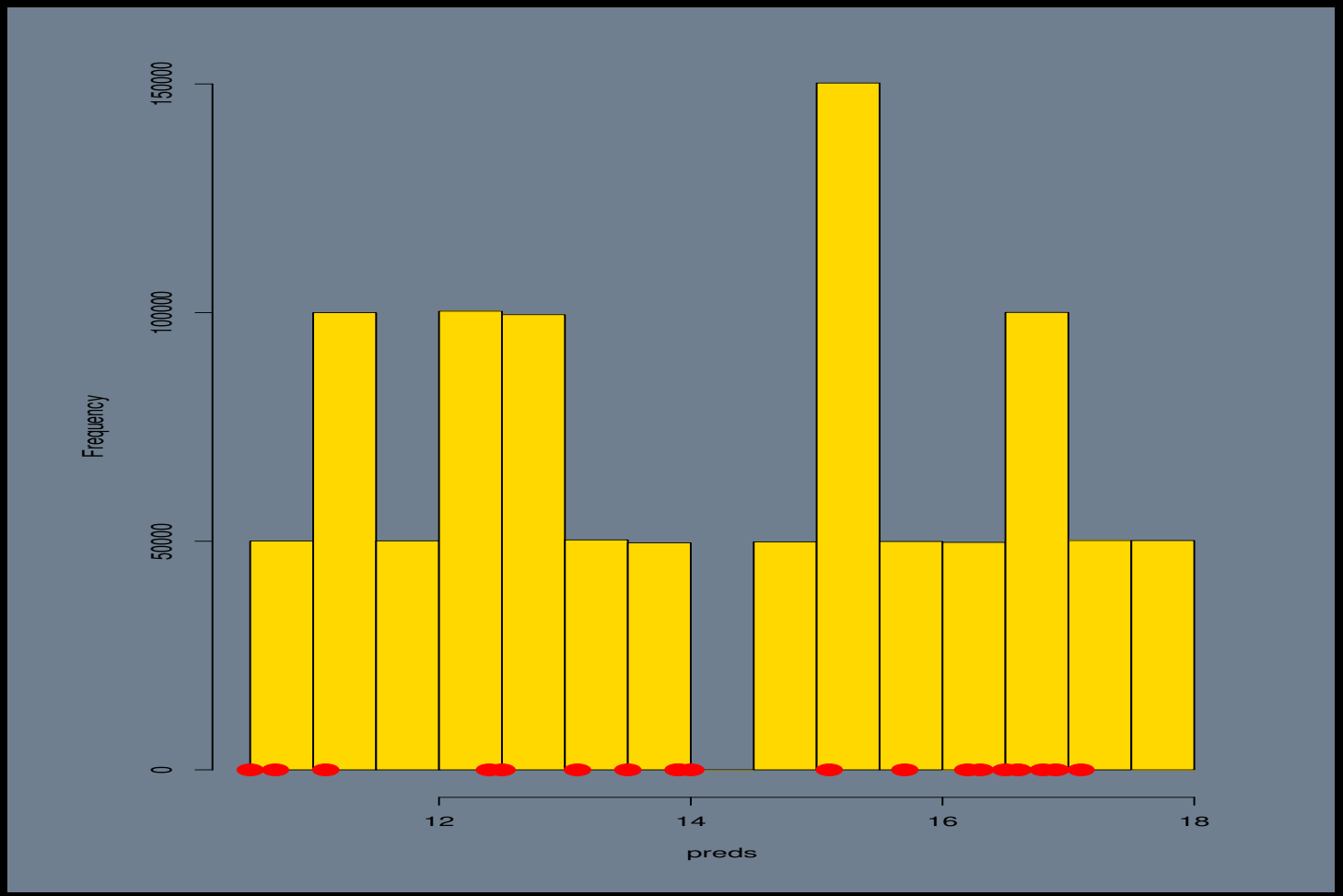
Looking at Predictions from Regression Models

- Often we can generate the consequences of making predictions from regression models in order to detect potential problems.

- Doing this with simulation has a lot of flexibility and is not a lot of work

```
anaemia <- read.table("http://jeffgill.org/data/anaemia.dat",
          header=TRUE,row.names=1)
anaemia.lm <- lm(Hb ~ PCV + Age + Menapause, data=anaemia)
preds <- sample(anaemia.lm$fitted.values,10^6,replace=TRUE)
par(mar=c(6,6,3,3),bg="slategrey",fg="black")
hist(preds,breaks=12,main="",col="gold")
points(anaemia$Hb,rep(1,length(anaemia$Hb)),col="red",cex=2,pch=19)
```

Looking at Predictions from Regression Models



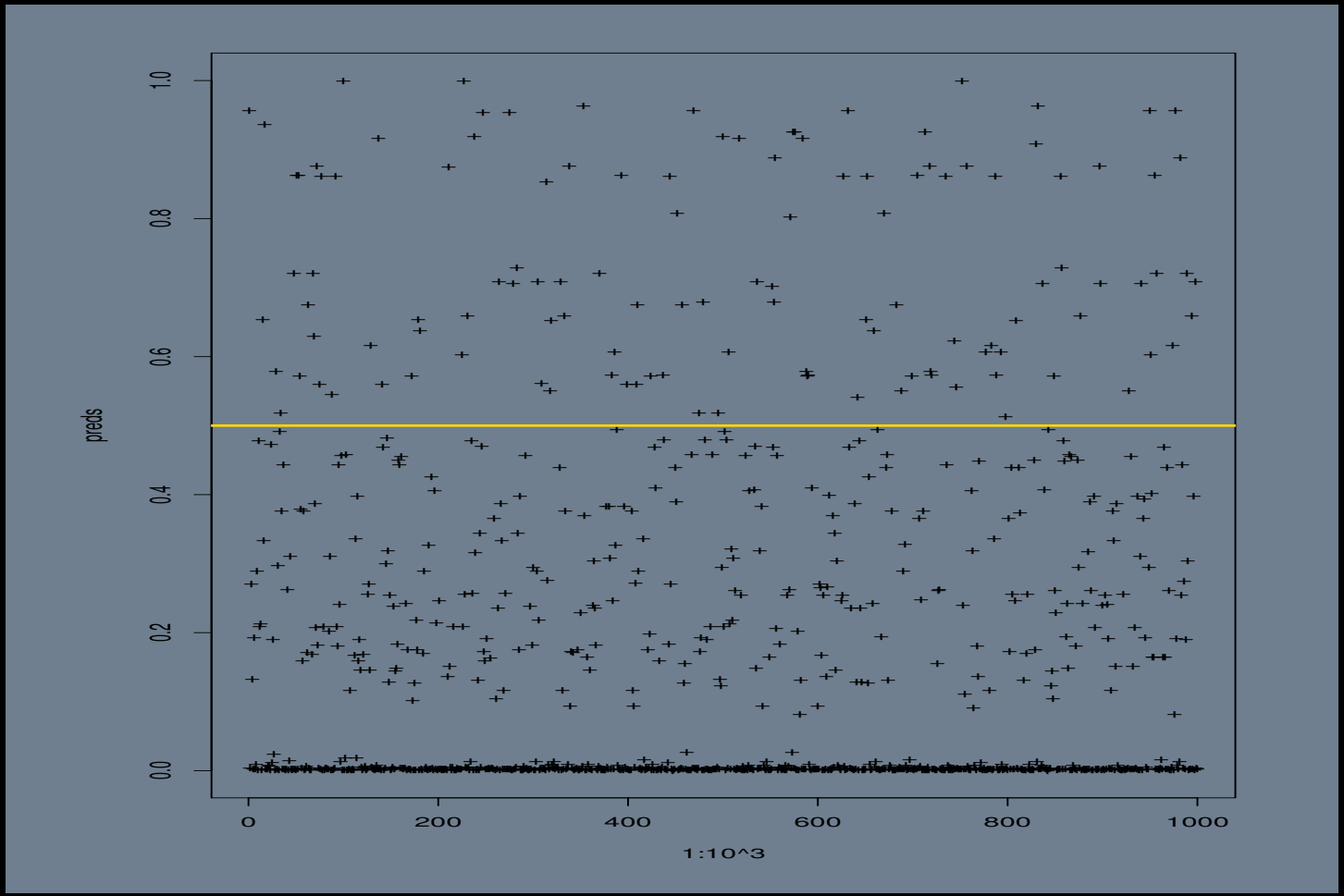
Looking at Predictions from Regression Models

Variables: Age: Age (Continuous), HG: Index of tumour stage and histolic grade (Continuous), Wt: Weight (Continuous), AP: Serum prostatic acid phosphatase C (Continuous), SBP: Systolic blood pressure (Continuous), PF: Performance rating (Categorical), DBP: Diastolic blood pressure (Continuous), HX: Cardiovascular disease history (Categorical), SG: Serum haemoglobin (Continuous), BM: Bone metastasis (Categorical), SZ: Size of primary tumour (Continuous), EKG: Electrocardiogram code (Categorical)

```
library(mgcv)
prostate.df <- read.table("http://jeffgill.org/data/prostate.full.dat",header=TRUE)
prostate.gam1 <- gam(bm ~ stage + pf + log1p(sz) + s(ap) + s(hg),
  family=binomial(link=logit), data=prostate.df)

preds <- sample(prostate.gam1$fitted.values,10^3,replace=TRUE)
par(mar=c(6,6,3,3),bg="slategrey",fg="black")
plot(1:10^3,preds,pch="+")
abline(h=0.5,lwd=3,col="gold")
```

Looking at Predictions from Regression Models



Basic Monte Carlo Integration

- Suppose $g(x)$ is difficult to express or manipulate but for which we could generate samples on an arbitrary support of interest: $[a{:}b]$.

- It is often the case where we can plug values into $g(x)$ but it is difficult to integrate, for example $g(x) = \exp[x]\left(\sin^{-1}(x) - 3x(3+2x^2)/(1-x^2)^{7/2}\right)$.

- A common quantity of interest is $$I[a,b] = \int_a^b g(x)h(x)dx,$$, that is, the expected value of some function, $h(x)$, of x distributed $g(x)$.

- A substitute for analytically calculating this is to randomly generate n values of x from $g(x)$ and calculate:
$$\hat{I}[a,b] = \frac{1}{n}\sum_{i=1}^{n} h(x_i).$$

- The idea is to replace analytical integration with summation from a large number of simulated values, rejecting (ignoring) values outside of $[a{:}b]$

Basic Monte Carlo Integration (cont).

- By the Strong Law of Large Numbers, $\hat{I}[a,b]$ converges with probability one to the desired value, $I[a,b]$ with an error rate that goes to zero proportional to $1/n^2$.

- More specifically $\hat{I}[a,b]$ now has "simulation error," this error is measured by the empirical variance of the simulation estimate:

$$Var(\hat{I}[a,b]) = \frac{1}{n(n-1)} \sum_{i=1}^{n} (h(x_i) - \hat{I}[a,b])^2$$

- Note that the researcher fully controls the simulation size, n, and therefore the simulation accuracy of the estimate.

Basic Monte Carlo Integration (cont).

- Because the central limit theorem applies here as long as $Var(I[a,b])$ is finite, credible intervals can be easily calculated by

$$[95\%_{lower}, 95\%_{upper}] = \left[\hat{I}[a,b] - 1.96\sqrt{Var(\hat{I}[a,b])},\ \hat{I}[a,b] + 1.96\sqrt{Var(\hat{I}[a,b])}\right],$$

  or by reporting the 0.025 and 0.075 quantiles of the set of $h(x_i)$.

- Bayesian context: replace $g(x)$ with a posterior statement $\pi(\theta|x)$ and $h(x)$ with $h(\theta)$. So $I[a,b]$ is really the posterior expectation of $h(\theta)$:

$$E[h(\theta)|x)] = \int \pi(\theta|x)h(\theta)d\theta \approx \frac{1}{n}\sum_{i=1}^{n} h(\theta).$$

Basic Monte Carlo Integration (cont).

- Example, standard normal PDF over a specific region, suppose we want:

$$I[-2,1] = \int_{-2}^{1} x^2 \phi(x) dx,$$

$$\hat{I}[-2,1] = \frac{1}{n} \sum_{i=1}^{n} x_i^2.$$

- In R

```
norm.sample <- rnorm(100000)
mean(norm.sample[norm.sample>-2 & norm.sample <1]^2)
[1] 0.5747261
```

- we can also "check" the algorithm with a related problem:

```
length(norm.sample[norm.sample>-2 & norm.sample <1])/100000
[1] 0.81989
(pnorm(2)-pnorm(0))+(pnorm(1)-pnorm(0))
[1] 0.8185946
```

Basic Monte Carlo Integration (cont).

- How about a "harder" problem:

$$I[e,\pi] = \int_e^{\pi} \arctan(x^{\frac{1}{3}}) \mathcal{C}(x|\theta = 3, \sigma = 2) dx$$

- Recall that:

$$\mathcal{C}(x|\theta,\sigma) = \frac{1}{\pi\sigma} \frac{1}{1 + \left(\frac{x-\theta}{\sigma}\right)^2}, \qquad -\infty < x, \theta < \infty, 0 < \sigma.$$

- R code:

```
c.sample <- rcauchy(100000,3,2)
mean(atan(c.sample[c.sample > exp(1) & c.sample < pi]^(1/3)))
[1] 1.058232
```

Rejection Sampling

- If we cannot produce random values from the posterior of interest, we cannot do MC integration as just described.

- New idea: rejection sampling according to. . .

  1. find some other distribution that is convenient to sample from,
  2. initially it must enclose the target distribution,
  3. compare generated values with PDF form and decide which values to keep and which to reject,
  4. integral of interest is the normalized ratio of these values.

- "Rejection sampling" used in two contexts: obtaining dimensional integral quantities, and generating random variables.

- Rejection sampling can be particularly useful in determining the normalizing factor for nonnormalized posterior distributions.

Continuous Form with Bounded Support

- Bounded support: defined, non-infinite range of some PDF or other function.
- Posit a function $f(x)$ such that $f(x)$ can be evaluated for any value over the support of x: $S_x = [A, B]$.
- Determine the maximum *density* value by either:
  1. analytical root finding,
  2. numerical techniques such as gridding, Newton-Raphson, EM, etc.
- Now we define a rectangle (or hyper-rectangle) that bounds all possible values for the pair $(x, f(x))$. Other geometric forms work too.
- Sample the two-dimensional random variable in this rectangle by independently sampling uniformly over $[A, B]$ and $[0, \tilde{x}]$ and pairing.
- Count the proportion of points that fall in the region of interest.

Continuous Form with Bounded Support (cont.)

- The value of the integral, the area under the curve is just the ratio of points under the curve to the total number of points scaled by the size of the box:

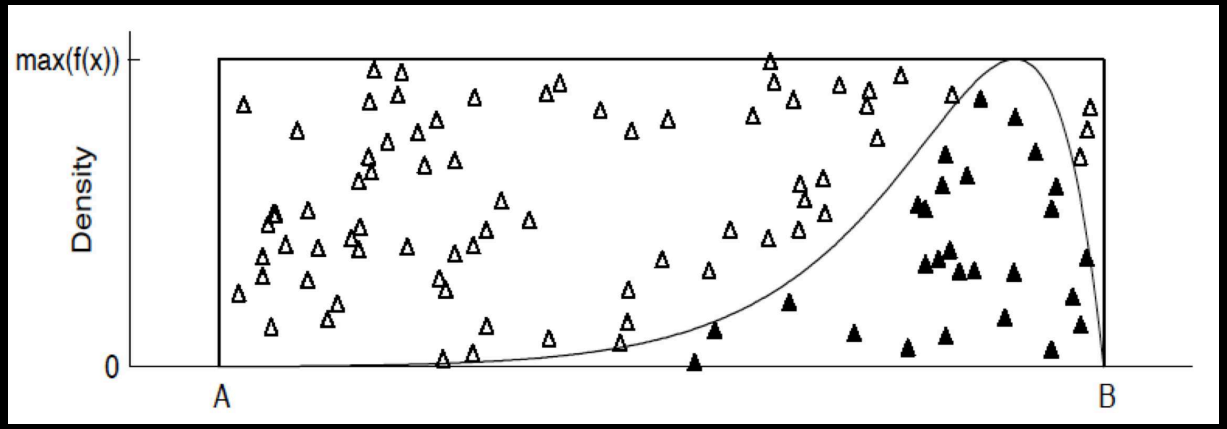
$$\frac{\text{number of points under curve}}{\text{total number of points}} \times \text{size of box} \underset{n\to\infty}{\longrightarrow} \int_A^B f(x)dx.$$

- Discrete problems turn out to be much more straightforward as it is usually a matter of counting bin heights and taking a weighted sum.

- The degree of accuracy is entirely controlled by the researcher through the number of points generated.

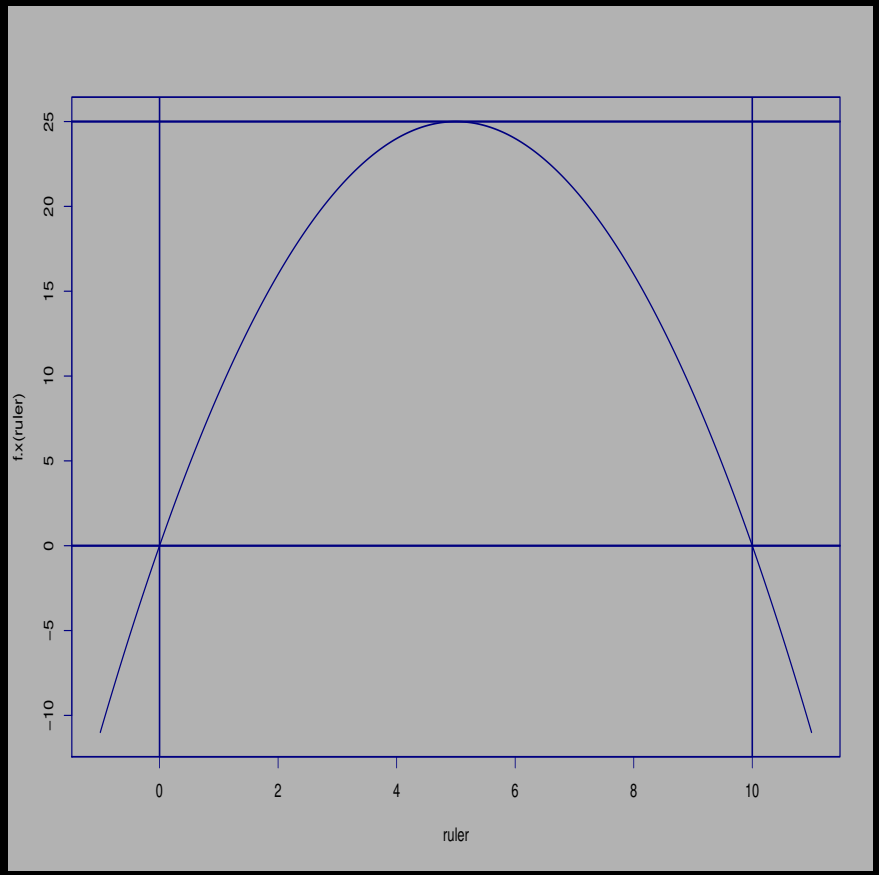
Continuous Form with Bounded Support (cont.)

- 100 points sampled uniformly from the two-dimensional rectangle over: $[(A, B), (0, \max(f(x)))] = [(0, 10), (0, 0.4)]$. In total 27 values fell into the area we wish to integrate, so we obtain the size of the interval from: $(27/100)(10 \times 0.4) = 1.04$.



Measuring the Area Under a Quadratic

- Question: what is the area under $f(x) = -x^2 + 3x + 7x$?
- This is not a difficult integral, but let's do it computationally.



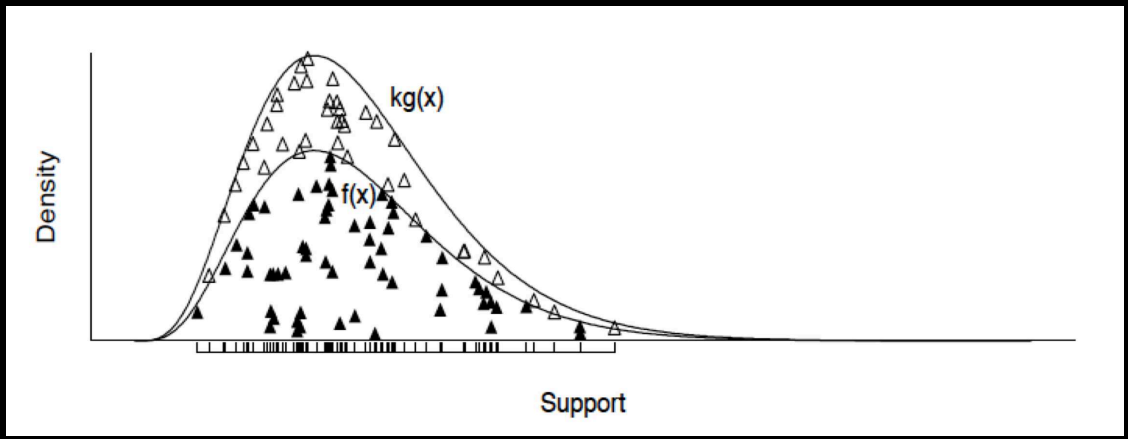
Measuring the Area Under a Quadratic

```
f.x <- function(x) -x^2 +3*x +7*x
ruler <- seq(-1,11,length=300)
par(bg="grey70",fg="navyblue",lwd=1.5)
plot(ruler, f.x(ruler),type="l")
abline(h=c(0,25),lwd=2)
abline(v=c(0,10),lwd=2)

n <- 100000
samples <- cbind(runif(n,0,10), runif(n,0,25))
10*25*sum(samples[,2]<f.x(samples[,1]))/n
[1] 166.0825
```

Continuous Form with Unbounded Support

- Now address the integral of some function $f(x)$ in which the analytical solution is difficult or impossible, and the form of $f(x)$ has unbounded tails.
- Specify a "majorizing function," $g(x)$, which for every value of x in the support of $f(x)$ has the property that $g(x) \geq f(x)$.



Continuous Form with Unbounded Support (cont.)

- If the target distribution has unbounded tails, then obviously the majorizing function must also have this property, and simply picking a PDF for $g(x)$ which has heavier tails than $f(x)$ will not work since it will then have other regions where it is not uniformly greater than the target.

- The solution is to use a multiplication factor so that:

$$f(x) \leq kg(x), \qquad \forall x, k > 1$$

- Sample x_i from $g(x)$ and then make an accept/reject decision based on $f(x_i)$.

Continuous Form with Unbounded Support (cont.)

- Steps:
  1. calculate $f(x_i)/kg(x_i)$,
  2. randomly draw a uniform(0,1) variate,
  3. accept x_i if this uniform is less than $f(x_i)/kg(x_i)$,
  4. repeat many times.

- If the majorizing distribution is very dissimilar from the target distribution, then the sampling procedure would be less efficient (more values rejected).

Example: Enveloping with an Exponential Density

- Start with nonnormalized folded normal kernel:

$$f(x) \propto \exp[-x^2/2], \qquad x \geq 0.$$

- In order to normalize this distribution, we require the integral quantity in the denominator of the right-hand side of Bayes' law:

$$I(x) = \int_0^\infty \exp[-x^2/2]dx$$

- Use an exponential enveloping distribution:

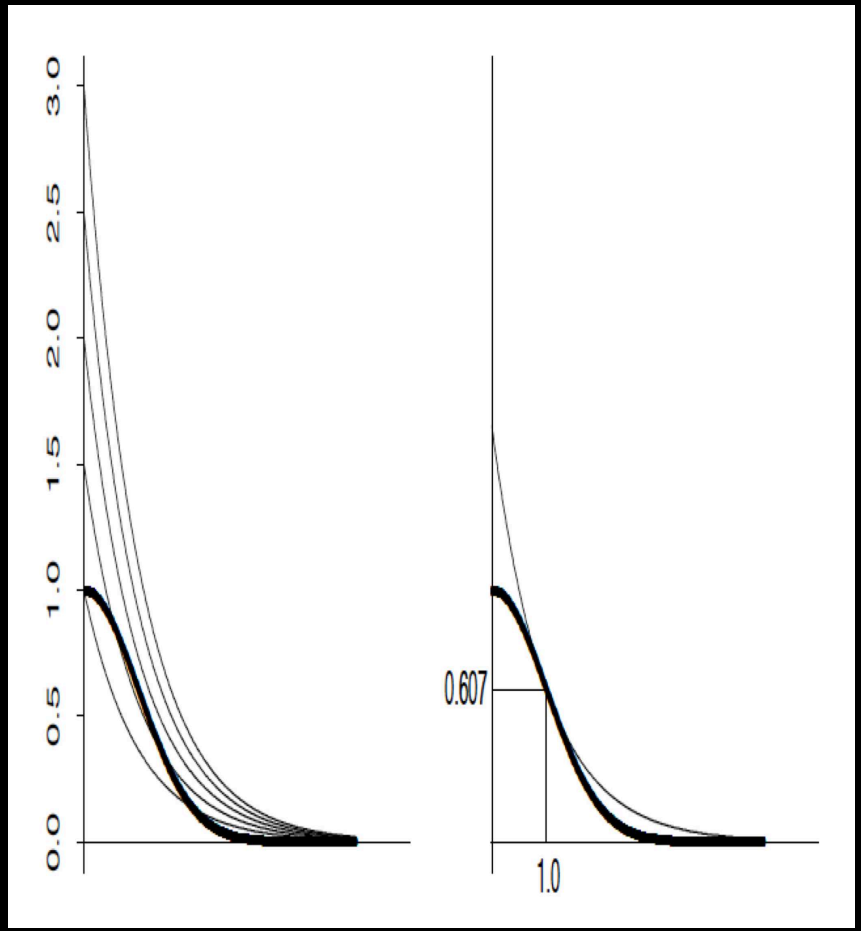
$$g(x) = k\exp[-x]$$

but what value of k is best to ensure coverage but maximize algorithmic efficiency by minimizing rejected values?

Example: Enveloping with an Exponential Density (cont.)

The left panel shows prospective exponential function along with the folded normal kernel (in bold). The exponential functions displayed are for $k = 1.0, 1.5, 2.0, 2.5, 3.0$ as indicated by the $x = 0$ point: $k\exp[0] = k$.

The right panel shows the optimal exponential enveloping function.



Example: Enveloping with an Exponential Density (cont.)

- To find the optimal coverage exponential, equate the two functions and solve for the k that gives the roots to the resulting quadratic equation:

$$g(x) \equiv k\exp[-x] = \exp[-x^2/2] \equiv f(x)$$
$$k = \exp[x - x^2/2]$$
$$0 = x^2 - 2x + 2\log k.$$

Equate the two possible quadratic solutions, one of which must be 1:

$$1 = 2\log k$$
$$\therefore k = \exp(1/2).$$

- Using this value, $k = 1.649$, in $g(x) = k\exp[-x]$, we get the enveloping function where the single point of intersection occurs at $[1, 0.607]$.

Example: Enveloping with an Exponential Density Steps

1. Draw n random variables from $\mathcal{EX}(1)$: $x_1, x_2, \ldots, x_n$.
2. For each x_i, calculate the corresponding value of the majorizing function: $kg(x_i) = (1.649)\exp[-x_i]$
3. Draw a random uniform value for each x_i over the interval from 0 to $kg(x_i)$: u_{x_i}.
4. Accept this draw as being from $f(x)$ if u_{x_i} is less than $f(x_i)$.
5. The number of accepted values relative to the total number of draws is proportional to the ratio of the area under the target function to the area under the majorizing function. So:

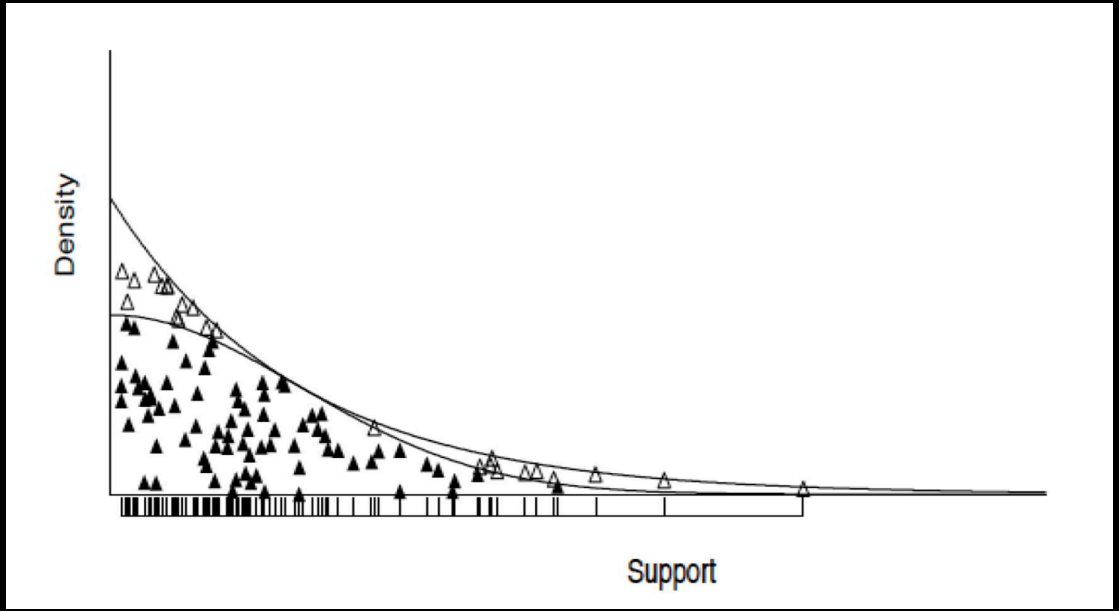
$$I(x) = \int_0^\infty f(x)dx = k \times \frac{\text{number of accepted points}}{\text{total number of draws}}$$

Example: Enveloping with an Exponential Density R Code

```
n <- 1000000
x.expo <- rexp(n, rate=1)
x.k.expo <- 1.649*exp(-x.expo)
u.vals <- rep(NA,n)
for (i in 1:n) u.vals[i] <- runif(1,0,x.k.expo[i])
f.x <- exp(-x.expo^2/2)
accepted <- 0
for (i in 1:n) if (u.vals[i] < f.x[i]) accepted <- accepted+1
1.649*accepted/n
[1] 1.252678
```

Example: Enveloping with an Exponential Density (cont.)

For only $n = 100$ draws, we get the reasonably accurate result of 1.2546, which is very close to the true value of 1.25314.

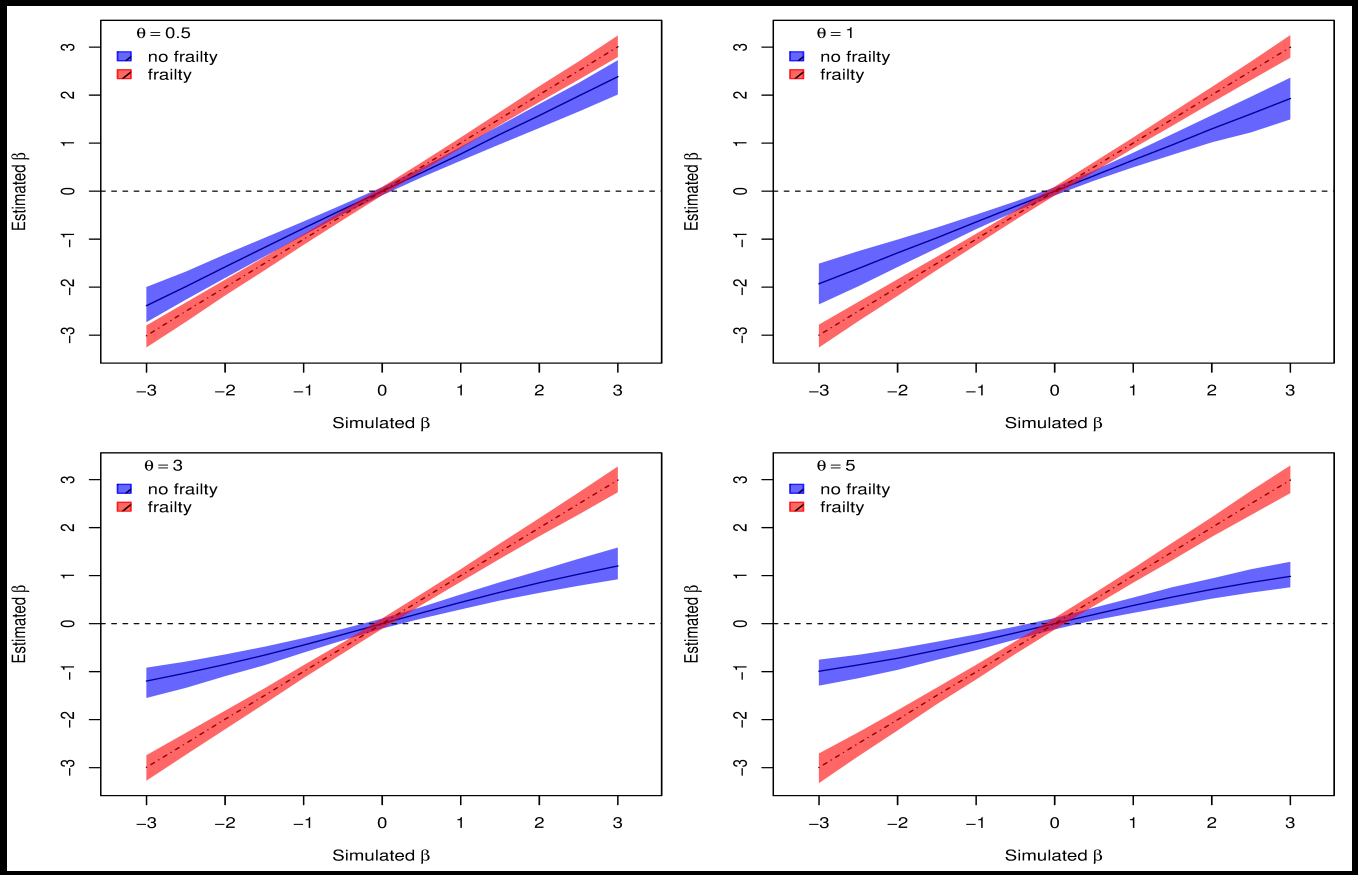


Regression Monte Carlo Example

- From Homola and Gill (2021 working paper).
- In order to illustrate the bias that arises when not accounting for heterogeneous multilevel effects (frailties) in the data, we run a series of Monte Carlo simulations.
- Specifically, we generated clustered survival data from a shared frailty model, with the usual hazard function specified by: $h(t_{ij}, X_{ij}) = h_0(t_{ij})\nu_j \exp(X_{ij}\boldsymbol{\beta})$, where ν_j, indicates the respective frailty value for group j, and follows a gamma distribution with mean 1 and variance θ.
- We simulated data for 20 groups with 25 observations per group, let x be normally distributed, and varied β (from -3 to 3).
- For each of these parameter combinations, we simulated 2,500 data sets and ran two survival models: one including the frailty parameter and one not including the frailty parameter.
- Data were generated for different values of β and θ using the `genfrail` function.

Regression Monte Carlo Example

- The 90% Monte Carlo confidence intervals for the point estimates of β are displayed in the figure.



Regression Monte Carlo Example

- For all four frailty specifications, we find that not accounting for the heterogeneous multilevel nature of the data leads to significantly biased coefficient estimates.
- The estimates for the non-frailty models are always substantially biased towards 0.
- For a gamma frailty distribution with mean 1 and variance 0.5, this bias is relatively modest (but significant), but as the variance of the frailty distribution increases, the bias grows rapidly.
- For example, if the frailty terms are distributed according to a gamma distribution with mean 1 and variance 5, then the point estimates of the non-frailty model for a simulated β value of 3 (or -3) are off by 2; a relative bias of 67%.
- Our simulations show the danger of not accounting for frailty effects when they are present in the data.
- Coefficient estimates will be severely biased towards 0.

What is a Stochastic Process?

♢ A type of stochastic process that will help us estimate posterior quantities.

♢ A *stochastic process* is a consecutive set of random quantities defined on some known state space, Θ, indexed so that the order is known: $\{\theta^{[t]} : t \in T\}$.

♢ Frequently, but not necessarily, T is the set of positive integers implying consecutive, even-spaced time intervals: $\{\theta^{[t=0]}, \theta^{[t=1]}, \theta^{[t=2]}, \ldots\}$.

♢ A stochastic process must also be defined with respect to a *state space*, Θ, which identifies the range of possible values of θ. This state space is either discrete or continuous depending on how the variable of interest is measured.

What is a Markov Chain?

♢ A *Markov chain* is a stochastic process with the property that any specified state in the series, $\theta^{[t]}$, is dependent only the previous value of the chain, $\theta^{[t-1]}$.

♢ Therefore values are *conditionally* independent of all other previous values: $\theta^{[0]}, \theta^{[1]}, \ldots, \theta^{[t-2]}$.

♢ Formally:

$$P(\theta^{[t]} \in A | \theta^{[0]}, \theta^{[1]}, \ldots, \theta^{[t-2]}, \theta^{[t-1]}) = P(\theta^{[t]} \in A | \theta^{[t-1]}),$$

where A is any identified set (an event or range of events) on the complete state space. (We will use this A notation extensively.)

♢ Colloquially:

> "A Markov chain wanders around the state space remembering only where it has been in the last period."

Why Is this Useful?

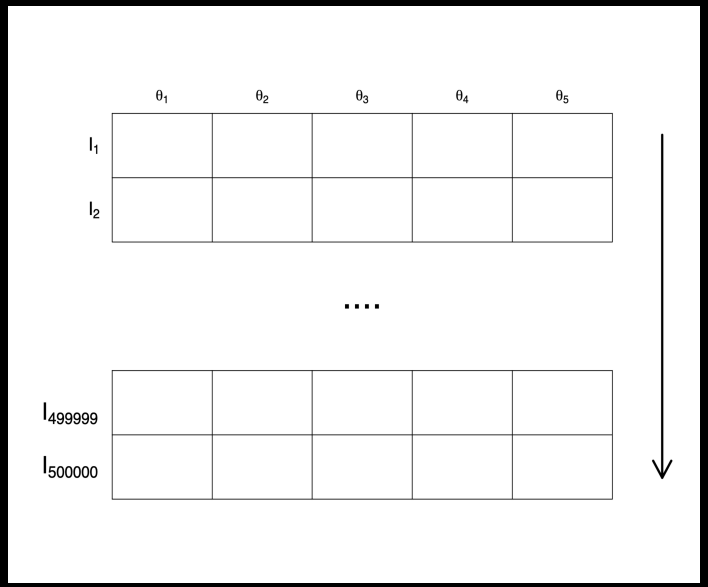
♢ This "short-term" memory property is very useful because when the chain eventually finds the region of the state space with highest density, it will wander around there producing a sample that is only modestly nonindependent.

♢ If this is the posterior region, then we can use these "empirical" values as legitimate posterior sample values.

♢ Thus difficult posterior calculations can be done with MCMC by letting the chain wander around "sufficiently long", thus producing summary statistics from recorded values.

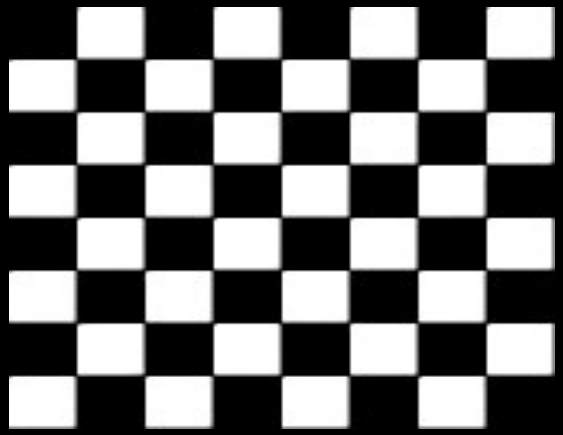
Returning to the Big Picture

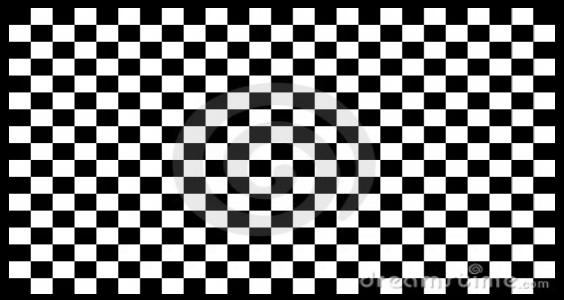
- So the Markov chain explores the multidimensional state space recording where it has been for us.
- This means that we can take the empirical values for each dimension (model parameter) and summarize them any way we want.

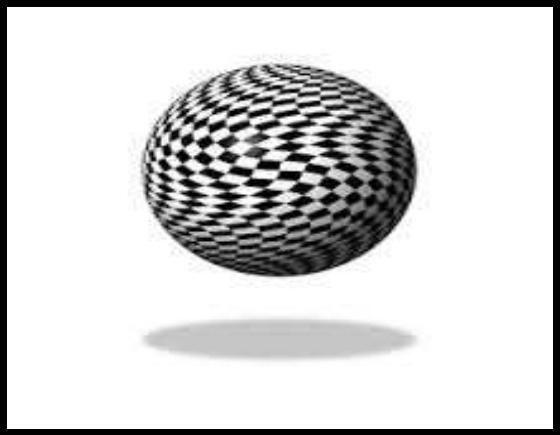


How Does it Move?

- How does the Markov chain decide to move?
- Define the *transition kernel*, K, as a general mechanism for describing the probability of moving to some other specified state based on the current chain status.
- $K(\theta, A)$ is a defined probability measure for all θ points in the state space to the set $A \in \Theta$.
- So $K(\theta, A)$ maps potential transition events to their probability of occurrence.







What is a Discrete Space Markov Chain Kernel?

♢ When the state space is discrete, K is a matrix mapping, $k \times k$ for k discrete elements in A, where each cell defines the probability of a state transition from the first term to all possible states:

$$P_A = \begin{bmatrix} p(\theta_1,\theta_1) & p(\theta_1,\theta_2) & \dots & p(\theta_1,\theta_{k-1}) & p(\theta_1,\theta_k) \\ p(\theta_2,\theta_1) & p(\theta_2,\theta_2) & \dots & p(\theta_2,\theta_{k-1}) & p(\theta_2,\theta_k) \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ p(\theta_k,\theta_1) & p(\theta_k,\theta_2) & \dots & p(\theta_k,\theta_{k-1}) & p(\theta_k,\theta_k) \end{bmatrix}$$

where the row indicates where the chain is at this period and the column indicates where the chain is going in the next period.

♢ Each matrix element is a well-behaved probability, $p(\theta_i,\theta_j) \geq 0,\ \forall i,j \in A$. When the state space is continuous, then K is a conditional PDF: $f(\theta|\theta_i)$.

♢ Rows of P_A sum to one and define a conditional PMF since they are all specified for the same starting value and cover each possible destination in the state space: for row i: $\sum_{j=1}^{k} p(\theta_i,\theta_j)$.

What is a Continuous Space Markov Chain Kernel?

♢ When the state space is continuous, then K is a conditional PDF: $f(\theta|\theta_i)$.

♢ K is a conditional PDF: $f(\theta|\theta_i)$, meaning a properly defined probability statement for all $\theta \in A$, given some given current state θ_i.

♢ Continuous state space Markov chains have more involved theory; so it's often convenient to think about discrete Markov chains at first.

What is a Markov Chain? (cont.)

♢ Transition probabilities between two selected states for arbitrary numbers of steps m can be calculated multiplicatively.

♢ The probability of transitioning from the state $\theta_i = x$ at time 0 to the state $\theta_j = y$ in exactly m steps is given by the multiplicative series:

$$p^m(\theta_i^{[0]} = x, |\theta_j^{[m]} = y) = \underbrace{\sum_{\theta_1}\sum_{\theta_2}\cdots\sum_{\theta_{m-1}}}_{\text{all possible paths}} \underbrace{p(\theta_i,\theta_1)p(\theta_1,\theta_2)\cdots p(\theta_{m-1},\theta_j)}_{\text{transition products}}.$$

♢ So $p^m(\theta_i^{[0]} = x, |\theta_j^{[m]} = y)$ is also a stochastic transition matrix that specifies the product of all the required intermediate steps where we sum over all possible paths that reach y from x.

A Two State Markov Chain

♢ A two-dimensional state space: a discrete vote choice between two political parties, a commercial purchase decision between two brands, etc.

♢ Voters/consumers/predators/viruses/etc. who normally select θ_1 have an 80% chance of continuing to do so, and voters/consumers who normally select θ_2 have only a 40% chance of continuing to do so.

♢ The transition matrix P:

$$\begin{array}{c} \\ \text{current period} \end{array} \begin{array}{c} \overbrace{}^{\text{next period}} \\ \begin{array}{cc} \theta_1 & \theta_2 \end{array} \\ \left\{ \begin{array}{c} \theta_1 \\ \theta_2 \end{array} \right. \end{array} \begin{bmatrix} 0.8 & 0.2 \\ 0.6 & 0.4 \end{bmatrix}.$$

A Two State Markov Chain

♢ Assign a starting point:

$$S_0 = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix};$$

♢ To get to the first state, we simply multiply the initial state by the transition matrix:

$$S_1 = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 0.8 & 0.2 \\ 0.6 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.7 & 0.3 \end{bmatrix} = S_1.$$

A Two State Markov Chain

♢ This series continues multiplicatively as long as we like:

$$\text{Second state:} \quad S_2 = \begin{bmatrix} 0.7 & 0.3 \end{bmatrix} \begin{bmatrix} 0.8 & 0.2 \\ 0.6 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.74 & 0.26 \end{bmatrix}$$

$$\text{Third state:} \quad S_3 = \begin{bmatrix} 0.74 & 0.26 \end{bmatrix} \begin{bmatrix} 0.8 & 0.2 \\ 0.6 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.748 & 0.252 \end{bmatrix}$$

$$\text{Fourth state:} \quad S_4 = \begin{bmatrix} 0.748 & 0.252 \end{bmatrix} \begin{bmatrix} 0.8 & 0.2 \\ 0.6 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.7496 & 0.2504 \end{bmatrix}.$$

A Two State Markov Chain

♢ The **Big Picture**: Imagine that this stationary distribution was the articulation of some PMF or PDF that we could not analytically describe but would like to. If we could run this Markov chain sufficiently long we would eventually get the stationary distribution *for any point in the state space*.

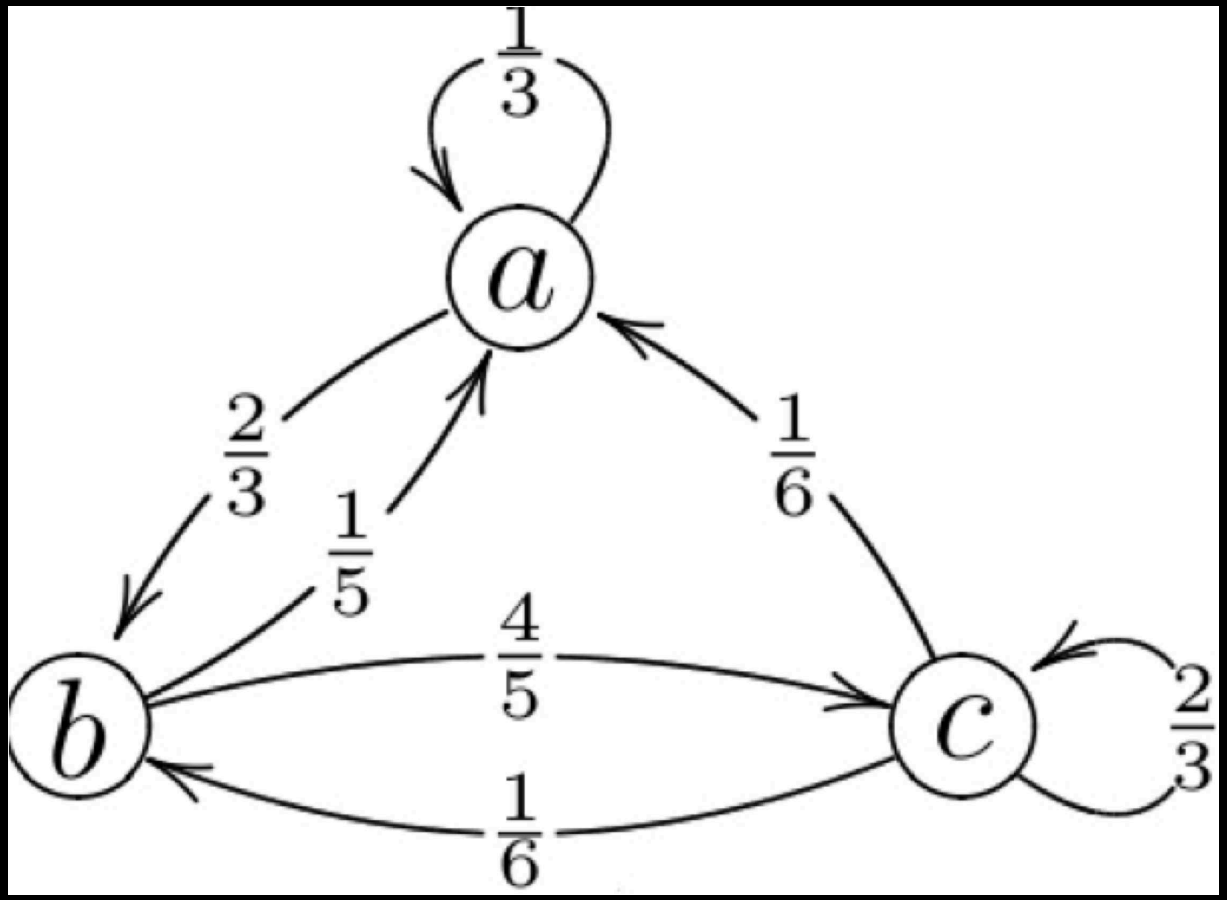
Actually, for this simple example we could solve directly for the steady state $S = [s_1, s_2]$ by stipulating:

$$\begin{bmatrix} s_1 & s_2 \end{bmatrix} \begin{bmatrix} 0.8 & 0.2 \\ 0.6 & 0.4 \end{bmatrix} = \begin{bmatrix} s_1 & s_2 \end{bmatrix},$$

and solving the resulting two equations for the two unknowns.

♢ This operation of running a Markov chain until it reaches its stationary distribution is exactly the process employed in MCMC.

Diagramatic Markov Chain



Another Simple Example

- Define the intended algorithm as some means of shuffling cards.
- The objective (stationary distribution) is a uniformly random distribution in the deck: for any given order to the stack, the probability of any one card occupying any one position is 1/52.
- Algorithm: take the top card and insert it uniformly randomly at some other point in the deck, continue.
- Is this a stochastic process?
- Is this a Markov chain?
- What is the limiting distribution?
- See BTW, Bayer and Diaconis (1992).

Another Simple Example

- Define the intended algorithm as some means of shuffling cards.
- The objective (stationary distribution) is a uniformly random distribution in the deck: for any given order to the stack, the probability of any one card occupying any one position is 1/52.
- Algorithm: take the top card and insert it uniformly randomly at some other point in the deck, continue.
- Is this a stochastic process?
- Is this a Markov chain?
- What is the limiting distribution?
- See BTW, Bayer and Diaconis (1992).

Another Simple Example (cont.)

- For simplicity (without loss of generality), set $n = 3$ cards.
- Sample space ($3!$ elements): $\{[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]\}$.
- Transition kernel:

$$P = \begin{bmatrix} 1/3 & 0 & 1/3 & 1/3 & 0 & 0 \\ 0 & 1/3 & 0 & 0 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 0 & 0 & 1/3 & 0 \\ 0 & 0 & 1/3 & 1/3 & 0 & 1/3 \end{bmatrix}$$

- Starting point: $[1,2,3]$.
- Assume no periodicity, obviously it is irreducible and closed (therefore positive reccurent), therefore this transition kernel defines an ergodic Markov chain.

Another Simple Example (cont.)

R code:

```
P <- matrix(c(1/3,0,1/3,0,1/3,0,0,1/3,1/3,0,1/3,0,1/3,0,1/3,0,0,1/3,
      1/3,0,0,1/3,0,1/3,0,1/3,0,1/3,1/3,0,0,1/3,0,1/3,0,1/3),nrow=6)
P
          [,1]      [,2]      [,3]      [,4]      [,5]      [,6]
[1,] 0.3333333 0.0000000 0.3333333 0.3333333 0.0000000 0.0000000
[2,] 0.0000000 0.3333333 0.0000000 0.0000000 0.3333333 0.3333333
[3,] 0.3333333 0.3333333 0.3333333 0.0000000 0.0000000 0.0000000
[4,] 0.0000000 0.0000000 0.0000000 0.3333333 0.3333333 0.3333333
[5,] 0.3333333 0.3333333 0.0000000 0.0000000 0.3333333 0.0000000
[6,] 0.0000000 0.0000000 0.3333333 0.3333333 0.0000000 0.3333333
```

Another Simple Example (cont.)

```
MC.multiply <- function(P.in,N) {
   S <- c(1,0,0,0,0,0)%*%P.in
   for (i in 2:N) {
     S <- S%*%P.in
     print(S)
   }
}

MC.multiply(P,15)

     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.2222222 0.1111111 0.2222222 0.2222222 0.1111111 0.1111111
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1851852 0.1481481 0.1851852 0.1851852 0.1481481 0.1481481
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1728395 0.1604938 0.1728395 0.1728395 0.1604938 0.1604938
```

```
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1687243 0.1646091 0.1687243 0.1687243 0.1646091 0.1646091
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1673525 0.1659808 0.1673525 0.1673525 0.1659808 0.1659808
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1668953 0.1664380 0.1668953 0.1668953 0.1664380 0.1664380
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1667429 0.1665905 0.1667429 0.1667429 0.1665905 0.1665905
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1666921 0.1666413 0.1666921 0.1666921 0.1666413 0.1666413
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1666751 0.1666582 0.1666751 0.1666751 0.1666582 0.1666582
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1666695 0.1666638 0.1666695 0.1666695 0.1666638 0.1666638
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1666676 0.1666657 0.1666676 0.1666676 0.1666657 0.1666657
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1666670 0.1666664 0.1666670 0.1666670 0.1666664 0.1666664
```

```
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1666668 0.1666666 0.1666668 0.1666668 0.1666666 0.1666666
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1666667 0.1666666 0.1666667 0.1666667 0.1666666 0.1666666
     [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
[1,] 0.1666667 0.1666666 0.1666667 0.1666667 0.1666666 0.1666666
```

The Chapman-Kolmogorov Equations

- Recall the probability of transitioning from the state $\theta_i = x$ at time 0 to the state $\theta_j = y$ in exactly m steps:

$$p^m(\theta_i^{[0]} = x, |\theta_j^{[m]} = y) = \underbrace{\sum_{\theta_1}\sum_{\theta_2}\cdots\sum_{\theta_{m-1}}}_{\text{all possible paths}} \underbrace{p(\theta_i,\theta_1)p(\theta_1,\theta_2)\cdots p(\theta_{m-1},\theta_j)}_{\text{transition products}}.$$

- These can be strung out to show how successive events are bound together probabilistically:

$$p^{m_1+m_2}(x,y) = \sum_{\text{all } z} p^{m_1}(x,z)p^{m_2}(z,y) \qquad \text{discrete case}$$

$$p^{m_1+m_2}(x,y) = \int_{\text{range } z} p^{m_1}(x,z)p^{m_2}(z,y)dz \qquad \text{continuous case.}$$

- The Chapman-Kolmogorov equations are particularly elegant for the discrete case because they can be represented as a series of transition matrix multiplications:

$$p^{m_1+m_2} = p^{m_1}p^{m_2} = p^{m_1}p^{m_2-1}p = p^{m_1}p^{m_2-2}p^2 = \ldots.$$

Marginal Distributions

- We want the *marginal* distribution at some step m th from the transition kernel.

- For the discrete case the marginal distribution of the chain at the m step is obtained by inserting the current value of the chain, $\theta_i^{[m]}$, into the row of the transition kernel for the mth step, p^m:

$$\pi^m(\theta) = [p^m(\theta_1), p^m(\theta_2), \ldots, p^m(\theta_k)].$$

- So the marginal distribution at the first step of discrete Markov chain is given by:

$$\pi^1(\theta) = p^1\pi^0(\theta),$$

where π^0 is the initial starting value assigned to the chain and $p^1 = p$ is a transition matrix.

Marginal Distributions in the Future

- The marginal distribution at some (possibly distant) step m for a given starting value is:

$$\pi^m = p\pi^{m-1} = p(p\pi^{m-2}) = p^2(p\pi^{m-3}) = \ldots = p^m\pi^0.$$

- Since successive products of probabilities quickly result in lower values, the property above shows how Markov chains eventually "forget" their starting points.

- The marginal distribution for the continuous case is only slightly more involved since we cannot just list as a vector the quantity:

$$\pi^m(\theta_j) = \int_\theta p(\theta, \theta_j)\pi^{m-1}(\theta)d\theta,$$

  which is the marginal distribution of the chain, given that it is currently on point θ_j at step m.

- Now we need a set of conditions that assures us that a given Markov chain will reach its stationary distribution, where the marginal distribution is constant and permanent.

Markov Chain Properties: Homogeneity

- *Homogeneity*
  - A Markov chain is said to be *homogeneous* at step m if the transition probabilities at this step do not depend on the value of m.
  - With non-homogeneous Markov chains the transition matrix is not constant but a function of time.
  - These are useful for modeling probabilistic decay processes, but not for our purposes.
  - The samplers that we use/pick are known to obtain the property, subject to machine implementation.
  - Counter-example: at the starting point a chain cannot be homogeneous since the marginal distribution for the first step is clearly not independent of the initial values that are hand-picked.

Markov Chain Properties: States

- *Some Characteristics of* States*:*
  - A state is *absorbing* if once the chain enters this state it cannot leave: $p(A, A^c) = 0$.
  - A state is *transient* if the probability of the chain not returning to this state is non-zero: $1 - p(A, A) > 0$. Also, phrases as *a finite number of visits in infinite time.*
  - State A is *closed* to state B if a Markov chain on A cannot reach B: $p(A, B) = 0$. State A is closed in the general sense if it is absorbing.

Markov Chain Properties: Irreducibility

- *Irreducibility*
  - A state is *irreducible* if for every θ_i and θ_j in this state, the two sub-states "communicate."
  - A Markov chain is *irreducible* on A if every reached point or collection of points (necessarily a subspace in the continuous case) can be reached from every other reached point or collection of points:

$$p(\theta_i, \theta_j) \neq 0, \ \forall \theta_i, \theta_j \in A.$$

  - A convenient way to remember the principle behind irreducibility is the notion that you could reduce the set if you wanted to, but that *you do not want to* because then there will be points that cannot be reached from other points.

Markov Chain Properties: Recurrence

- *Recurrence*
  - *If a state is closed, discrete or continuous but finite, and irreducible, then this state and all sub-spaces within this subspace are recurrent.*
  - Markov chains operating on recurrent state spaces are recurrent.
  - Recurrence is good!
  - Formal definition:
    A irreducible Markov chain is called *recurrent* with regard to a given state, A, which is a single point or a defined collection of points (required for the bounded-continuous case), if the probability that the chain occupies A infinitely often over unbounded time is nonzero.

Markov Chain Properties: Recurrence

- *Recurrence*
  - Colloquial definition:
    When a chain moves into a recurrent state, it stays there forever and visits every subspace infinitely often.
  - A Markov chain is *positive recurrent* if the mean time to return to A is bounded.
  - Otherwise it is called *null recurrent*.
  - Note: irreducible chains are either recurrent or transient.

Markov Chain Properties: Harris Recurrence

- *Harris Recurrence*
  - If we only had to deal with discrete or finite state spaces, then standard recurrence would be enough.
  - With unbounded-continuous state spaces it is necessary to have a stricter definition that guarantees that the probability of visiting subspace A infinitely often in the limit is still one.
  - Define η_A as the number of visits to A in the limit.
  - Require $P(B) > 0 \, \forall \, B \subset A$.
  - The set A is Harris recurrent if $P(\eta_B = \infty) = 1, \quad \forall B \in A$.
  - An irreducible Markov chain is Harris recurrent if it every possible subspace is Harris recurrent.
  - An aperiodic, irreducible chain with an invariant distribution on an unbounded continuous state space that is *not* Harris recurrent has a positive probability of getting stuck forever in an area bounded away from convergence, given a starting point there.

Markov Chain Properties: Linkages

- *Recurrence and Irreducibility*
  - The union of a set of recurrent states (nonempty, and bounded or countable) or Harris recurrent states is a new state that is closed and irreducible.
  - The linkage between recurrence and irreducibility is important in defining a subspace that captures a Markov chain and at the same time assures that this Markov chain will explore all of the subspace.
  - **Important result:**
    - Given a Markov chain on continuous state space,
    - when the chain wanders into a closed, irreducible set of Harris recurrent states,
    - it stays there forever and visits every single sub-state (region) with probability one.

Markov Chain Properties: Stationarity

- *Stationarity*
  - Define $\pi(\theta)$ as the stationary distribution of the Markov chain for θ on the state space A.
  - Recall that $p(\theta_i, \theta_j)$ is the probability that the chain will move from θ_i to θ_j at some arbitrary step t,
  - and $\pi^t(\theta)$ is the corresponding marginal distribution.
  - The stationary distribution satisfies:

$$\sum_{\theta_i} \pi^t(\theta_i) p(\theta_i, \theta_j) = \pi^{t+1}(\theta_j) \qquad \text{Discrete case}$$

$$\int \pi^t(\theta_i) p(\theta_i, \theta_j) d\theta_i = \pi^{t+1}(\theta_j) \qquad \text{Continuous case.}$$

    meaning that application of the kernel to the current distribution probabilities returns the same probabilities.

Markov Chain Properties: Stationarity

- The marginal distribution remains fixed when the chain reaches the stationary distribution and we might as well drop the superscript designation for iteration number and just use $\pi(\theta)$.
- In shorthand: $\pi = \pi p$.
- Once the chain reaches its stationary distribution, it stays in this distribution and moves about, or "mixes," throughout the subspace according to marginal distribution, $\pi(\theta)$, forever.
- Recurrence gives the range restriction property whereas stationarity gives constancy of the probability structure that dictates movement.

Markov Chain Properties: Periodicity

- *Periodicity*
  - The *period* of a Markov chain is the length of time required to repeat an identical cycle of chain values.
  - Actually, periodicity is bad and we want an *aperiodic* Markov chain: the repeating length is the trivial value of one.
  - The problem with periodicity is that it destroys the probabilistic property of the transition kernel.
  - Update of our status: we have recurrence to make sure that we revisit states infinitely often in the limit, we have stationarity to make sure that eventually we will obtain constancy of the probability structure, but we don't have any guarantee about what kind of distribution we obtain in this limit.
  - Caveat: all Markov chains run on computers are actually periodic ("Anyone who considers arithmetical methods of producing random digits is, of course, in a state of sin." –John von Neumann [1903-1957])

Markov Chain Properties: Ergodicity

- If a chain is positive Harris recurrent (unbounded continous case) or recurrent (discrete/bounded continouus case), and aperiodic, then we call it *ergodic*. Ergodic Markov chains have the property:

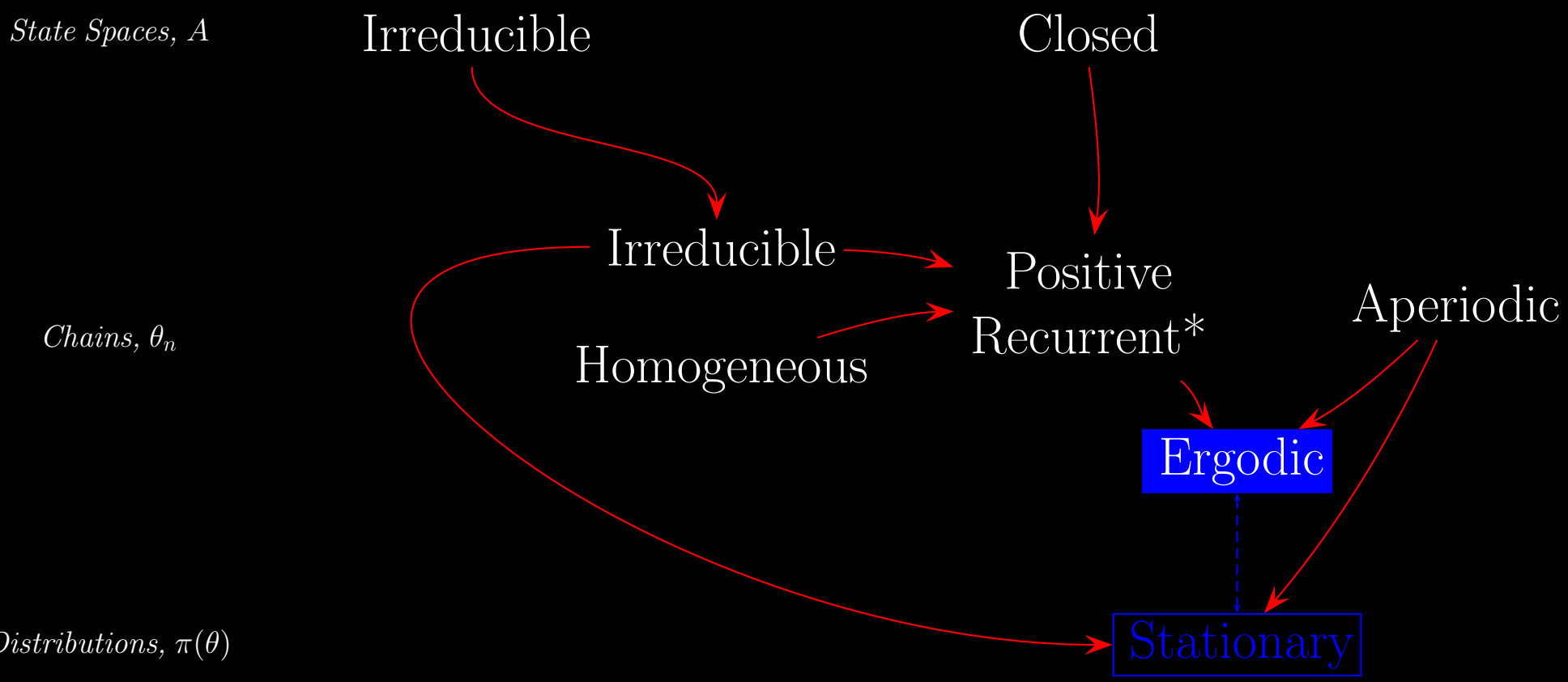
$$\lim_{n\to\infty} K^n(\theta_i, \theta_j) = \pi(\theta_j),$$

  for all θ_i, and θ_j in the subspace

- What this means: *once a specified chain is determined to have reached its ergodic state, sample values behave as if they were produced by the posterior of interest from the model.*

- The *ergodic theorem* is the equivalent of the strong law of large numbers but for Markov chains, since it states that any specified function of the posterior distribution can be estimated with samples from a Markov chain in its ergodic state because averages of sample values give strongly consistent parameter estimates.

Markov Chain Properties



*Discrete or bounded continuous: $E[\eta_A] = \infty \; \forall \theta \in A$. Continuous: $P(\theta_n \in A) = 1, \; \forall A \in H$ where $P > 0$ (Harris). Positive recurrence: mean return time to A is bounded.

Markov Chain Properties

- *Consequences of Ergodicity*
  - Suppose $\theta_{i+1}, \ldots, \theta_{i+n}$ are n (not necessarily consecutive) values from a Markov chain that has reached its ergodic distribution, a statistic of interest, $h(\theta)$, can be calculated empirically:

$$\hat{h}(\theta_i) = \frac{1}{n} \sum_{j=i+1}^{i+n} h(\theta_j) = \int h(\theta) d\pi(\theta)$$

    This is the ergodic theorem.

  - For finite quantities this converges almost surely:

$$p[\hat{h}(\theta_i) \rightarrow h(\theta), \text{ as } n \rightarrow \infty] = 1$$

  - Even though Markov chain values, by their very definition, have serial dependence, the mean of the chain values provides a strongly consistent estimate of the true parameter.

Markov Chain Properties

- *Flavors of Ergodicity*
  - Nothing yet said about the *rate of convergence*.
  - **Geometric Ergodicity**: a Markov chain with stationary distribution π at time t, a positive constant $r \in (0, 1)$, and a non-negative, real-valued function of θ such that:

$$||P(\theta^{[t]}) - \pi|| \leq f(\theta) r^t.$$

  - **Uniform Ergodicity**: replace $f(\theta)$ with a positive constant k:

$$||\hat{h}(\theta_i) - h(\theta)|| \leq k r^t.$$

  - Where

$$\|f(\boldsymbol{\theta}_t) - \pi(\boldsymbol{\theta})\| = \frac{1}{2} \sup_{\boldsymbol{\theta} \in A} A \int_{\boldsymbol{\Theta}} |f(\boldsymbol{\theta}_t) - \pi(\boldsymbol{\theta})| \, d\boldsymbol{\theta},$$

    which is also half of the L_1 distance (the $1/2$ is sometimes omitted by authors).

Markov Chain Properties

- *Flavors of Ergodicity (cont.)*
  - No matter which flavor of ergodicity applies, we get the important theoretical result from the CLT.
  - For a given empirical estimator $\hat{h}(\theta_i)$ with bounded limiting variance, we get:

$$\sqrt{n}\frac{\hat{h}(\theta_i) - h(\theta)}{\sqrt{Var(\hat{h}(\theta_i))}} \underset{n\to\infty}{\longrightarrow} \mathcal{N}(0,1)$$

  since $\hat{h}(\theta_i) = \frac{1}{n}\sum_{j=i+1}^{i+n} h(\theta_j)$ is really a mean.

Getting Started with the BUGS Language

- Models in the BUGS language are specified more theoretically than in other packages.
- We will follow G&H and use vague priors everywhere, but this is an important issue in some instances.
- Example from the assigned reading for next week: "Bayesian Analytical Methods: A Methodological Prescription for Public Administration."
- Data from the 1998 and 2004 rounds of the American State Administrator's Project (ASA) survey.
- Our outcome variable of interest `grp.influence` is an index of the respondents' (senior executives). perceptions of the influence that clientele groups have on the total agency budget, specialized program budgets, and agency policies. Each of these questions is a seven-point scale and they have been summed to create a single outcome variable (ranging from 3 to 21).

Homework Exercise

- Explanatory variables: `contracting`, `gov.influence`, `leg.influence`, `elect.board`, `years.tenure`, `education`, `party.ID`, `category2`, `category3`, `category4`, `category5`, `category6`, `category7`, `category8`, `category9`, `category10`, `category11`, `category12`, `med.time`, `medt.contr`, `gov.ideology`, `lobbyists`, `nonprofits`.

- Data for `JAGS` needs to be in list form:

```
asap.jags.list <- list(STATES <- 50, SUBJECTS <- 713,
  state.id <- c(1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,...),
  contracting <- c(6, 2, 0, 0, 0, 1, 0, 3, 3, 6, 0, 1, 5, 1, 1, 1, 0,...),
  :
  nonprofits <- c(1.9783, 0.509, 2.0701, 1.3639, 15.6682, 2.7968,...)
)
```

- Or read this in from a file that you've constructed.

Setting Up the Markov Chain

- Markov chains require starting points, so create the following to modify:

```
"tau" <- 1
"tau.alpha" <- 1
"alpha" <- c(0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
      0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
      0,0,0,0,0,0,0,0,0,0,0,0)
"beta" <- c(0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0)
"gamma" <- c(0,0,0)
```

- Then make them different.

The First Part of the JAGS Code

```
model {
 for (i in 1:SUBJECTS) {
  mu[i] <- alpha[state.id[i]]
   + beta[1]*contracting[i] + beta[2]*gov.influence[i] + beta[3]*leg.influence[i]
   + beta[4]*elect.board[i] + beta[5]*years.tenure[i] + beta[6]*education[i]
   + beta[7]*party.ID[i]  + beta[8]*category2[i]  + beta[9]*category3[i]
   + beta[10]*category4[i] + beta[11]*category5[i]  + beta[12]*category6[i]
   + beta[13]*category7[i] + beta[14]*category8[i]  + beta[15]*category9[i]
   + beta[16]*category10[i] + beta[17]*category11[i]  + beta[18]*category12[i]
   + beta[19]*med.time[i]  + beta[20]*medt.contr[i]
  grp.influence[i] ~ dnorm(mu[i],tau)
 }
 for (j in 1:STATES) {
  eta[j] <- gamma[1]*gov.ideology[j] + gamma[2]*lobbyists[j]
      + gamma[3]*nonprofits[j]
  alpha[j] ~ dnorm(eta[j],tau.alpha)
 }
```

The Second Part of the JAGS Code

```
 beta[1]  ~ dnorm(0.070,1)   # PRIOR MEANS FROM KELLEHER AND YACKEE 2009, MODEL 3
 beta[2]  ~ dnorm(-0.054,1)
 beta[3]  ~ dnorm(0.139,1)
 beta[4]  ~ dnorm(0.051,1)
 beta[5]  ~ dnorm(0.017,1)
 beta[6]  ~ dnorm(0.056,1)
 :
 beta[18] ~ dnorm(0.0,1)    # DIFFUSE PRIORS
 beta[19] ~ dnorm(0.184,1)   # PRIOR MEANS FROM KELLEHER AND YACKEE 2009, MODEL 3
 beta[20] ~ dnorm(0.156,1)
 gamma[1] ~ dnorm(0.0,1)    # DIFFUSE PRIORS
 gamma[2] ~ dnorm(0.0,1)
 gamma[3] ~ dnorm(0.0,1)
 tau    ~ dgamma(1.0,1)
 tau.alpha ~ dgamma(1.0,1)
}
```

Running JAGS From R

```
# LOAD LIBRARY AND SOURCE FILES
library(rjags); library(arm); library(coda); library(superdiag)

# DEFINE THE MODEL
asap.model2.rjags <- function() {
 for (i in 1:SUBJECTS) {
 :
}

# SAVE MODEL TO A FILE
write.model(asap.model2.rjags, "Article.JPART/asap.model2.rjags")
```

Running JAGS From R

```
# RUN THE SAMPLER AND COLLECT coda SAMPLES
asap2.model <- jags.model(file="Article.JPART/asap.model2.rjags",
  inits=asap.inits, data=asap.jags.list, n.chains=3, n.adapt=5000)
update(asap2.model, n.iter=2500)
asap2.mcmc <- coda.samples(model=asap2.model, variable.names=names(asap.jags.list),
  n.iter=2500)
summary(asap2.mcmc)

# CHECK CONVERGENCE
superdiag(as.mcmc.list(asap2.mcmc), burnin=0)

# GET THE DEVIANCE AND THE DIC
asap2.dic <- dic.samples(asap2.model, n.iter=25000, type="pD")
```